

Visual Workflow Automation Platform — Build Blueprint

- [PROJECT NAME TBD] — Visual Workflow Automation Platform
- 0. One-Paragraph Pitch
 - 1. Complete Functional Requirements (build against these, don't drift)
 - 2. Scope: MVP vs. Later
 - 3. Technology Stack (fixed decisions — don't substitute without a reason)
 - 4. High-Level Architecture
 - 5. Repository Structure (create this exact skeleton in Phase 1)
 - 6. Data Model
 - 7. API Surface (v1)
 - 8. Auth & Authorization
 - 9. Credential Management
 - 10. Workflow Execution Engine
 - 11. Node System
 - 12. Celery + Redis Execution Architecture
 - 13. Triggers
 - 14. Frontend Editor Structure
 - 15. Live Execution Updates (WebSocket)
 - 16. Integrations Architecture
 - 17. AI / Agent Architecture (post-MVP, Phase 12)
 - 18. Error Handling, Retries & Idempotency
 - 19. Security Checklist
 - 20. Observability
 - 21. Testing Strategy
 - 22. Docker & Environments
 - 23. PHASE-BY-PHASE BUILD PLAN
 - 24. MVP "Definition of Done" (final checklist)
 - 25. Roadmap Beyond MVP

[PROJECT NAME TBD] — Visual Workflow Automation Platform

An n8n-style automation tool — Full Build Blueprint (FastAPI Edition)

How to use this document: This is a self-contained specification. Paste this entire file into any AI coding assistant (Claude, GPT, Cursor, etc.) as project context, then ask it to implement one phase at a time. Each phase below lists exact deliverables, files to create, and acceptance criteria so the AI always knows “what does done look like.”

Name candidates (pick one before Phase 1, or ask your AI to help brand it): Fluxpoint, Nodeflow, Wireloom, Circuitry, Pathwire, Flowsmith, Relaygraph, Loomwork

0. One-Paragraph Pitch

A self-hosted, visual, drag-and-drop workflow automation platform (n8n-style). Users build directed graphs of **trigger → logic → action → AI** nodes on a canvas. Workflows run manually, via webhooks, or on cron schedules. A FastAPI control plane accepts requests and persists state to PostgreSQL; Celery workers (via Redis) execute workflows asynchronously so the API is never blocked; a WebSocket channel streams live execution status back to the browser.

Golden rule for every phase: FastAPI handles requests. Celery workers execute workflows. The API must never block on workflow execution.

1. Complete Functional Requirements (build against these, don’t drift)

ID	Requirement
FR-01	Auth — register, login, logout; users only see workspaces they belong to
FR-02	Workflow management — create/edit/save/activate/deactivate/duplicate/delete; drafts vs. published versions are distinct and versioned
FR-03	Visual editor — drag nodes onto canvas, configure via side panel, connect ports, pan/zoom/move, undo/redo, live graph validation
FR-04	Execution — create an execution record → enqueue Celery job → run nodes in dependency order → persist per-node status/input/output
FR-05	Triggers — manual trigger, webhook trigger, timezone-aware cron schedule
FR-06	Expressions — reference upstream node outputs via a <i>controlled</i> (non-Python, AST-based, allowlisted) expression language, e.g. <code>{{ \$node.HTTP_Request.body.amount }}</code>
FR-07	Reliability — retries, timeouts, cancellation, failure policy, idempotency keys for external side effects
FR-08	Credentials — encrypted at rest (AES-GCM or KMS), referenced only by ID, redacted from all logs

Non-functional requirements: API stays responsive during long workflows · workers scale independently of the API · critical state is durable (Postgres, not Redis) · all external input validated (Pydantic) · secrets never touch logs or the frontend · entire stack is reproducible via Docker Compose.

2. Scope: MVP vs. Later

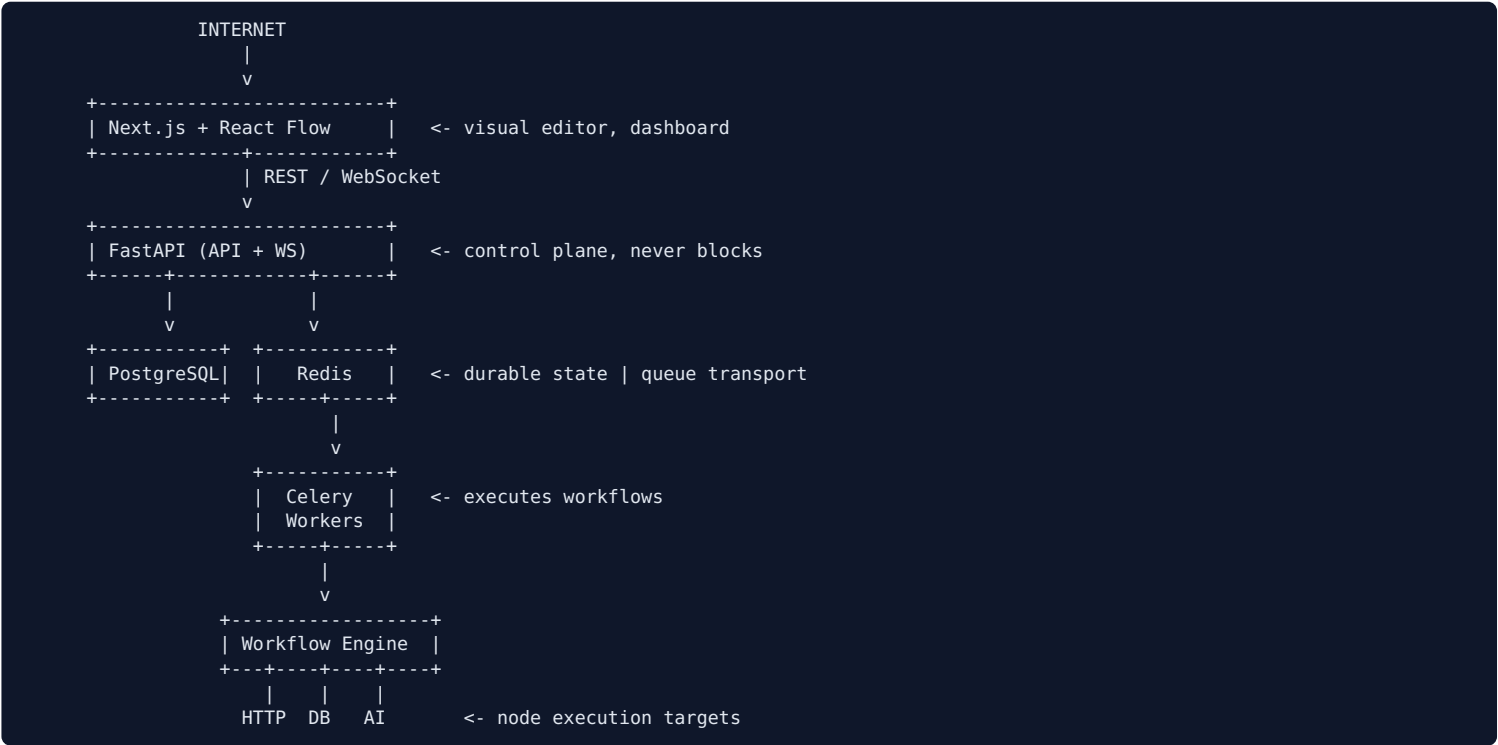
Area	MVP (build first)	Later (post-MVP)
Auth	Register / login / protected APIs	OAuth / SSO / SCIM
Editor	Canvas, nodes, connections, config panel	Real-time collaboration, comments
Triggers	Manual, webhook, schedule	Integration-native event triggers
Nodes	HTTP Request, Set, IF, Switch, Delay	Merge, loops, sub-workflows
Execution	Celery, status tracking, logs, retry	Priority queues, per-tenant rate limits
Integrations	HTTP + PostgreSQL + Slack + Gmail	Large integration marketplace
AI	Optional single LLM node	Multi-step agents, RAG, AI-generated workflows

Rule for the AI building this: do not start Phase 12 (AI/agents) before Phases 1-11 are solid. Deterministic execution must be proven reliable first.

3. Technology Stack (fixed decisions — don’t substitute without a reason)

Layer	Technology	Purpose
Frontend	Next.js + React + TypeScript	Dashboard + editor shell
Canvas	React Flow (@xyflow/react)	Node/edge graph editor
UI kit	Tailwind CSS + shadcn/ui	Component styling
Client state	Zustand	Editor state management
Backend	FastAPI (Python)	REST API + WebSockets
Validation	Pydantic v2	Request/response schemas
ORM	SQLAlchemy 2.0 (async)	Database access
Migrations	Alembic	Schema versioning
Database	PostgreSQL	Durable source of truth
Queue broker	Redis	Task transport
Task queue	Celery	Background execution
Scheduler	Celery Beat	Cron-based dispatch
Realtime	FastAPI native WebSockets	Live execution status
Storage	S3-compatible (or Cloudflare R2)	Large artifacts
Testing	Pytest (backend) + Playwright (E2E)	Automated tests
Deployment	Docker + GitHub Actions	Reproducible builds/CI

4. High-Level Architecture



Request lifecycle:

Browser → FastAPI → validate request → create execution row
→ enqueue Celery task → return execution_id (API returns immediately)
→ Celery worker runs workflow → PostgreSQL stores per-node results
→ worker publishes events → FastAPI WebSocket pushes to browser
→ browser updates live execution overlay

5. Repository Structure (create this exact skeleton in Phase 1)

```

orchestrix/                                # rename to chosen project name
├── apps/
│   ├── web/                               # Next.js frontend
│   │   ├── app/
│   │   ├── components/
│   │   ├── features/
│   │   ├── workflow-editor/
│   │   │   ├── canvas/
│   │   │   ├── nodes/
│   │   │   ├── edges/
│   │   │   ├── config-panel/
│   │   │   ├── toolbar/
│   │   │   └── execution-overlay/
│   │   ├── store/                         # Zustand stores
│   │   └── lib/
│   └── backend/                           # FastAPI service
│       ├── app/
│       │   ├── main.py
│       │   ├── api/v1/                    # route handlers (thin!)
│       │   ├── core/                     # config, security, settings
│       │   ├── db/                       # SQLAlchemy engine/session, Alembic
│       │   ├── models/                   # ORM models
│       │   ├── schemas/                  # Pydantic schemas
│       │   ├── services/                 # business logic
│       │   ├── workflow_engine/          # graph execution logic
│       │   ├── nodes/                    # node implementations
│       │   ├── workers/                  # Celery tasks
│       │   ├── queue/                    # Redis/Celery config
│       │   ├── integrations/             # Slack, Gmail, GitHub, Sheets...
│       │   └── websocket/                # live event streaming
│       └── tests/
├── infrastructure/                       # IaC, deployment configs
├── scripts/
├── docker-compose.yml
├── .env.example
└── README.md

```

Layering rule for the AI: Route → dependency (auth/workspace check) → service → repository/DB. Routes must stay thin — no business logic in `api/v1/`.

6. Data Model

6.1 Workflow Graph JSON (the canvas serializes to this)

```

{
  "id": "wf_123",
  "name": "Lead Automation",
  "version": 3,
  "status": "active",
  "nodes": [
    {
      "id": "trigger_1",
      "type": "webhook",
      "position": { "x": 100, "y": 200 },
      "config": {}
    },
    {
      "id": "http_1",
      "type": "http-request",
      "position": { "x": 400, "y": 200 },
      "config": {
        "method": "GET",
        "url": "https://api.example.com/leads/{{trigger_1.body.id}}"
      },
      "credentialId": "cred_456"
    }
  ],
  "connections": [
    { "source": "trigger_1", "sourcePort": "main", "target": "http_1", "targetPort": "main" }
  ]
}

```

Hard rule: credentials are referenced by `credentialId` only — never embedded as plaintext in workflow JSON.

6.2 PostgreSQL Tables

Table	Purpose
users	Identity
workspaces	Tenant/workspace boundary
workspace_members	RBAC membership
workflows	Workflow metadata (current pointer)
workflow_versions	Immutable versioned definitions
executions	Execution history (one row per run)
execution_nodes	Per-node status/input/output/error
credentials	Encrypted secret payloads
webhooks	Inbound trigger registrations
schedules	Cron expression + timezone
templates	Reusable workflow definitions
api_keys	Programmatic access tokens
audit_logs	Security/activity history

Required indexes: - workflows(workspace_id, updated_at) - workflow_versions(workflow_id, version) - executions(workflow_id, started_at) - execution_nodes(execution_id) - webhooks(path, method) - audit_logs(workspace_id, created_at)

7. API Surface (v1)

Method	Endpoint	Purpose
POST	/api/v1/auth/register	Register
POST	/api/v1/auth/login	Login
POST	/api/v1/auth/logout	Logout
GET	/api/v1/workflows	List workflows
POST	/api/v1/workflows	Create workflow
GET	/api/v1/workflows/{id}	Get workflow
PUT	/api/v1/workflows/{id}	Update workflow
POST	/api/v1/workflows/{id}/activate	Activate
POST	/api/v1/workflows/{id}/execute	Trigger manual execution
GET	/api/v1/executions/{id}	Execution detail
POST	/api/v1/executions/{id}/cancel	Cancel a running execution
GET	/api/v1/credentials	List credential metadata (never secret values)
POST	/api/v1/credentials	Create credential
POST	/api/v1/webhooks/{workflow_id}	External trigger endpoint
GET	/api/v1/templates	List templates
WS	/ws/executions/{execution_id}	Live execution events

Base app skeleton:

```
from fastapi import FastAPI

app = FastAPI(title="[Project Name] API", version="1.0.0")
app.include_router(auth_router, prefix="/api/v1/auth")
app.include_router(workflow_router, prefix="/api/v1/workflows")
app.include_router(execution_router, prefix="/api/v1/executions")
app.include_router(webhook_router, prefix="/api/v1/webhooks")

@app.get("/health")
async def health():
    return {"status": "ok"}
```

8. Auth & Authorization

- FastAPI dependencies enforce authentication on every protected route.
- Enforce **workspace membership** before touching any workspace-scoped resource.
- Roles: `Owner`, `Admin`, `Editor`, `Viewer`.
- Every workflow / execution / credential query must be filtered by the caller's current `workspace_id` — no exceptions, no “trust the client-sent `workspace_id`.”

9. Credential Management

```
Client → TLS → FastAPI → validate
                        |
                        v
                    encrypt secret
                        |
                        v
                PostgreSQL (stores ciphertext only)
                        |
                credentialId only is returned to client
                        |
                        v
                  Worker (at execution time)
                        |
                decrypt in memory (never persisted decrypted)
                        |
                        v
                Node / external API call
```

- Use authenticated encryption (AES-GCM) or a managed secrets service.
- Encryption keys live **outside** the database (env var / KMS / vault).
- Redact secret values from all logs, error messages, and execution history output.

10. Workflow Execution Engine

Execution lifecycle: `QUEUED → RUNNING → SUCCEEDED / FAILED / CANCELED / TIMED_OUT` **Node lifecycle:** `PENDING → RUNNING → SUCCESS / FAILED / SKIPPED`

```
class WorkflowEngine:
    async def execute(self, workflow, trigger_data):
        graph = self.validator.validate(workflow)
        ctx = ExecutionContext(trigger_data=trigger_data)
        ready = graph.initial_nodes()
        while ready:
            node = ready.pop()
            result = await self.executor.run(node, ctx)
            ctx.store(node.id, result)
            ready.extend(graph.next_nodes(node, result))
        return ctx
```

Graph validation must catch: duplicate node IDs, missing connection targets, invalid ports, unreachable required nodes, illegal cycles, missing required config, incompatible port connections.

11. Node System

```
from pydantic import BaseModel

class HttpConfig(BaseModel):
    method: str
    url: str
    headers: dict[str, str] = {}
    body: dict | None = None

class HttpRequestNode:
    type = "http-request"

    async def execute(self, ctx, config: HttpConfig):
        url = ctx.resolve(config.url)
        response = await ctx.http.request(
            config.method, url, headers=config.headers, json=config.body
        )
        return {"main": [response.json()]}
```

MVP node set: Manual Trigger, Webhook, Schedule, HTTP Request, Set, IF, Switch, Delay, PostgreSQL, Slack, Email. Add AI nodes **only after** deterministic execution is stable (see \$2 rule).

Expression Language

```
{{ $json.email }}
{{ $node.Webhook.body.customerId }}
{{ $node.HTTP_Request.body.amount }}
{{ $vars.environment }}
```

Implement with a **controlled AST parser + allowlisted functions**. Never expose raw Python `eval`, filesystem access, process environment, or unrestricted network calls through expressions — this is a security boundary, not a nice-to-have.

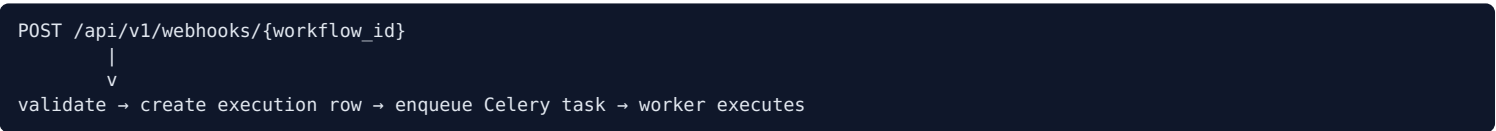
12. Celery + Redis Execution Architecture



```
@celery_app.task(
    bind=True,
    autoretry_for=(TransientError,),
    retry_backoff=True,
    max_retries=3,
)
def execute_workflow(self, execution_id):
    return run_execution(execution_id)
```

Use bounded worker concurrency, retries with backoff, and graceful shutdown handling. **The FastAPI process itself must never run a workflow inline.**

13. Triggers



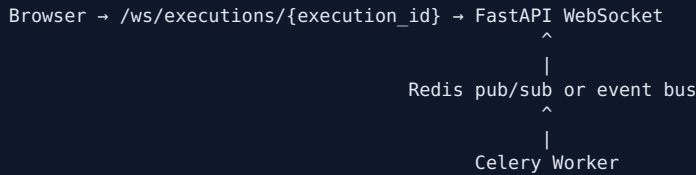
- Schedules store a cron expression + explicit timezone.
- Celery Beat **dispatches** jobs on schedule; it does not execute the workflow logic itself — that’s still the worker’s job.

14. Frontend Editor Structure

```
workflow-editor/
├─ canvas/           # React Flow canvas surface
├─ nodes/            # custom node renderers
├─ edges/            # custom edge renderers
├─ config-panel/     # per-node configuration form
├─ toolbar/          # save/activate/undo/redo/zoom controls
└─ execution-overlay/ # live status pills on nodes during a run
```

Editor responsibilities: draft graph state, node configuration, live validation, save/load, “test execution,” and live execution status overlaid directly on the canvas via the WebSocket feed.

15. Live Execution Updates (WebSocket)



Events to emit: `execution.started`, `node.started`, `node.completed`, `node.failed`, `execution.completed`, `execution.failed`. Every event payload includes `executionId`, `nodeId`, `timestamp`, and only safe/redacted diagnostic metadata.

16. Integrations Architecture

Start with **HTTP Request** — it unlocks arbitrary REST APIs with zero extra adapter code. Dedicated adapters (Slack, Gmail, etc.) each own: credential schema, client, config schema, response mapping, rate-limit behavior, retry classification, and secret redaction.

```
integrations/
├── slack/
│   ├── client.py
│   ├── schemas.py
│   └── service.py
├── gmail/
├── github/
└── google_sheets/
```

17. AI / Agent Architecture (post-MVP, Phase 12)

```
AI Node
├──
├-- Provider abstraction (OpenAI / Gemini / Anthropic / local Ollama)
├-- Tools (HTTP / Database / Search)
├-- Structured Output
└-- Guardrails
```

Agent controls to implement: max iterations, timeout, allowlisted tools, structured-output schema enforcement, optional human-approval gate, and usage/cost metadata capture.

18. Error Handling, Retries & Idempotency

Failure class	Example	Policy
Validation	Missing config	No retry
Authentication	Expired token	Refresh if supported
Rate limit	HTTP 429	Backoff + retry
Timeout	Network timeout	Bounded retry
Provider 5xx	Service unavailable	Bounded retry
Provider 4xx	Bad request	Usually fail immediately
Internal	Unexpected exception	Fail + alert

- Exponential backoff **with jitter**.
- For any external side effect (send email, create record, charge card, etc.), derive an **idempotency key** from `(execution_id, node_id, logical_operation)`.
- Never blindly retry a non-idempotent operation.

19. Security Checklist

- TLS everywhere
- Secure password hashing (argon2/bcrypt)
- Protected session/token handling
- Workspace-scoped authorization on every query
- Rate limiting on public endpoints (esp. webhooks)

- Strict CORS policy
- Secret redaction in logs and execution history
- Pydantic validation on all external input
- Secure response headers
- **SSRF protection is mandatory** for the HTTP node: block private/loopback IP ranges, resolve and re-check destination IPs, enforce response size/time limits, route through an egress proxy if possible
- **Code nodes never execute arbitrary user Python inside the main FastAPI/Celery process.** A real “Code” node needs full sandbox isolation (CPU/memory/filesystem/network limits) — treat this as a separate, later, carefully-scoped feature.

20. Observability

- **Logs:** application + worker errors
- **Metrics:** queue depth, execution count, execution duration, failure rate, worker utilization
- **Traces:** request → queue → worker → node, using one correlation/execution ID end to end
- **Audit logs:** workflow activation/deletion, credential changes, membership changes

21. Testing Strategy

Test type	Coverage
Unit	Graph traversal, expression parser, validators, node logic
API	Auth, CRUD, authorization boundaries, webhooks
Integration	PostgreSQL, Redis/Celery, integration adapters
Worker	Task lifecycle, retry, timeout, cancellation
E2E	Create → activate → trigger → inspect execution result
Security	Authz bypass attempts, SSRF, secret leakage
Load	Concurrent webhook + execution traffic

Minimum acceptance workflow to test against: Webhook → HTTP Request → IF → PostgreSQL/Slack . Must verify: enqueueing, worker execution, persisted history, live UI updates, and failure handling.

22. Docker & Environments

```
docker compose
├─ postgres
├─ redis
├─ backend
├─ celery-worker
├─ celery-beat
└─ web
```

Production topology: load balancer/reverse proxy → Next.js + multiple FastAPI instances; Redis → multiple Celery workers; managed PostgreSQL; object storage for large artifacts.

.env.example:

```
DATABASE_URL=postgresql+asyncpg://user:password@localhost:5432/[project_name]
REDIS_URL=redis://localhost:6379/0
CELERY_BROKER_URL=redis://localhost:6379/1
CELERY_RESULT_BACKEND=redis://localhost:6379/2
SECRET_KEY=replace-with-long-random-secret
CREDENTIAL_ENCRYPTION_KEY=replace-with-strong-key
FRONTEND_URL=http://localhost:3000
API_URL=http://localhost:8000
OPENAI_API_KEY=
GEMINI_API_KEY=
ANTHROPIC_API_KEY=
```

Local dev commands:

```
# Backend
cd apps/backend
python -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
cp .env.example .env
alembic upgrade head
uvicorn app.main:app --reload --port 8000

# Worker
celery -A app.workers.celery_app worker --loglevel=info --concurrency=4

# Scheduler
celery -A app.workers.celery_app beat --loglevel=info

# Frontend
cd apps/web
pnpm install
pnpm dev
```

Use Docker Compose for local Postgres/Redis. Commit only `.env.example`, never real secrets.

23. PHASE-BY-PHASE BUILD PLAN

This is the exact order to hand to an AI coding agent, one phase per session/PR. Each phase has a **definition of done**.

Phase 1 — Foundations

- Scaffold the monorepo structure from §5.
- FastAPI app boots with `/health`.
- PostgreSQL + SQLAlchemy async engine + Alembic configured, first migration runs.
- Redis running via Docker Compose.
- `docker-compose.yml` brings up postgres + redis + backend.
- **Done when:** `docker compose up` gives a working FastAPI + Postgres + Redis stack with `/health` returning 200.

Phase 2 — Auth & Workspace Model

- `users`, `workspaces`, `workspace_members` tables + migrations.
- Register / login / logout endpoints, password hashing, session/JWT tokens.
- Auth dependency + workspace-membership dependency for protected routes.
- **Done when:** a user can register, log in, and hit a protected `/workflows` stub scoped to their workspace only.

Phase 3 — Workflow CRUD & Versioning

- `workflows`, `workflow_versions` tables.
- Create/list/get/update/duplicate/delete endpoints.
- Draft vs. published version distinction; activate/deactivate endpoint.
- **Done when:** full CRUD + versioning works via API tests, no editor UI needed yet.

Phase 4 — React Flow Editor (frontend)

- Next.js app with React Flow canvas, node palette, drag-to-add.
- Config panel for editing a selected node's config JSON.
- Zustand store for draft graph state; save/load wired to Phase 3 API.
- Undo/redo, pan/zoom, basic client-side graph validation.
- **Done when:** a user can visually build the graph in §6.1 and save it.

Phase 5 — Python Workflow Engine (no queue yet)

- Implement `WorkflowEngine`, `ExecutionContext`, graph validator (§10).
- Implement MVP nodes: Manual Trigger, HTTP Request, Set, IF, Switch, Delay (§11).
- Implement the expression parser (§11) with allowlisted functions only.
- Run engine synchronously in a test harness (not yet via Celery).
- **Done when:** the engine can execute the minimum acceptance workflow (§21) end-to-end in a test, in-process.

Phase 6 — Celery Workers & Execution History

- `executions`, `execution_nodes` tables.
- Celery + Redis wired (§12); `execute_workflow` task calls the Phase 5 engine.

- `/workflows/{id}/execute` enqueues instead of running inline.
- Persist per-node status/input/output/error to `execution_nodes`.
- **Done when:** hitting `execute` returns instantly with an `execution_id`, and a worker log shows the workflow actually ran and history is queryable via `/executions/{id}`.

Phase 7 — Triggers: Manual / Webhook / Schedule

- `webhooks`, `schedules` tables.
- `/api/v1/webhooks/{workflow_id}` public endpoint → validates → creates execution → enqueues.
- Celery Beat wired for cron schedules with timezone support.
- **Done when:** the minimum acceptance workflow can be triggered by curl-ing the webhook URL and by a scheduled cron tick.

Phase 8 — WebSocket Execution UI

- `/ws/executions/{execution_id}` FastAPI WebSocket endpoint.
- Redis pub/sub bridging worker events → WebSocket (§15 event list).
- Frontend `execution-overlay/` shows live per-node status on the canvas.
- **Done when:** triggering a run shows nodes light up green/red live in the browser without a page refresh.

Phase 9 — Credentials & Security Hardening

- `credentials` table + AES-GCM encryption service (§9).
- Credential CRUD endpoints (metadata only ever returned).
- SSRF protections on the HTTP node (§19).
- Secret redaction in logs/execution history.
- Idempotency key support for external side effects (§18).
- **Done when:** a credential can be created, referenced by ID in a workflow, decrypted only inside a worker at execution time, and never appears in any log or API response.

Phase 10 — Real Integrations

- Implement `integrations/postgresql/`, `integrations/slack/` (and optionally `gmail/`) per §16.
- Each adapter: credential schema, client, response mapping, rate-limit + retry classification.
- **Done when:** the minimum acceptance workflow can actually post to a real Slack channel or write to a Postgres table using stored credentials.

Phase 11 — Testing, Observability, Production Docker

- Fill out the test matrix in §21 (unit/API/integration/worker/E2E/security/load).
- Add structured logging, basic metrics (queue depth, execution duration, failure rate), and a correlation ID threaded through request → queue → worker → node.
- Production-grade `docker-compose.yml` / Dockerfiles; GitHub Actions CI running tests + build.
- **Done when:** the §22 “Definition of Done” MVP checklist is fully green with automated tests passing in CI.

Phase 12 — AI Node & Agents (post-MVP)

- Only start after Phase 11 is stable.
- Provider abstraction (OpenAI/Gemini/Anthropic/Ollama), tool-calling (HTTP/DB/Search), structured output, guardrails (§17).
- Agent controls: max iterations, timeout, allowlisted tools, optional human approval, usage/cost tracking.
- **Done when:** an AI node can be dropped into a workflow, call an LLM with a structured schema, and its output is usable by downstream nodes like any other node result.

24. MVP “Definition of Done” (final checklist)

- ☐ Auth: register/login/logout, workspace-scoped access
- ☐ Workflow editing: canvas + graph persistence + versions
- ☐ Manual execution works
- ☐ Webhook execution works
- ☐ Scheduled execution works
- ☐ Celery workers execute workflows off the request path
- ☐ PostgreSQL execution history is queryable per node
- ☐ WebSocket live status updates render on canvas
- ☐ Credentials encrypted at rest, decrypted only in-worker, redacted from logs

- ☐ Retries/timeouts/idempotency implemented
- ☐ At least one real integration works end-to-end (Slack or Postgres)
- ☐ Automated tests cover unit/API/integration/E2E
- ☐ Full stack runs via `docker compose up`

Final demo flow:

```
Create Workflow → Webhook → HTTP → IF → PostgreSQL/Slack → Activate
→ Send Webhook → FastAPI → Redis → Celery Worker → Workflow Engine
→ Execution History + Live Status + Errors visible in UI
```

25. Roadmap Beyond MVP

OAuth integrations catalog, workflow marketplace/templates, sub-workflows, human-approval steps, real-time multi-user collaboration, workflow version diffing, enterprise SSO, usage metering/billing, AI-generated workflows, AI-assisted debugging, RAG/vector database nodes, multi-region execution.

This document is an independent architecture blueprint inspired by n8n’s product concept. It is not official n8n documentation and contains no n8n source code.